A Project Report

on

Detection of Alphanumeric Handwritten Values using Fuzzy Logic

*carried out as part of the **Soft Computing Techniques (INT3141)** Submitted*

by

Abhijay Shrivastava
2430030333

Aarush Dayal
2430030303

Section: Soft Computing D

in partial fulfilment for the award of the degree of

Bachelor of Technology

in

Information Technology



MANIPAL UNIVERSITY
JAIPUR

Department of Information Technology

MANIPAL UNIVERSITY JAIPUR
RAJASTHAN, INDIA

Oct 2026

Code repository: https://github.com/Abhijay09/Softcomputing-project

1. Introduction

Handwritten character recognition is one of the application areas named in the course hand-out. Handwriting is imprecise: the same letter written twice never looks identical, and different letters (O/0, S/5, I/1) can look alike. Such imprecision cannot be handled well by crisp yes/no rules, which is exactly the situation fuzzy logic was designed for (CO2). In this project we build a system that watches a live camera feed (laptop or USB camera), finds handwritten characters, and recognises each of them with a fuzzy rule base. For every character the system also reports how reliable the reading is, computed by a second small fuzzy controller.

Course outcomes addressed: INT3141.2 (apply fuzzy logic and reasoning) and INT3141.5 (develop and test a soft computing technique on a real-life problem).

2. Formal Problem Description

2.1 Input

A video frame is a colour image of H × W pixels (480 × 640 for the laptop camera), each pixel having a red, green and blue value from 0 to 255.

The frame contains n ≥ 0 handwritten characters written in dark ink on a light background. For training we use a labelled set of N grey-scale character images g of size 28 × 28, each with a label y from the alphabet A.

Alphabet: A is the set of 47 EMNIST-balanced classes – the digits 0–9, the upper-case letters A–Z (some letters that look the same in both cases, e.g. C/c, O/o, S/s, are one class) and the lower-case letters a, b, d, e, f, g, h, n, q, r, t.

2.2 Output

An ordered sequence of n pairs, where c is the recognised character at position k (reading order: left to right, top to bottom) and ρ is its fuzzy reliability in percent:

$$(c_1, \rho_1),\ (c_2, \rho_2),\ \ldots,\ (c_n, \rho_n)$$

The text output is the recognised characters joined into one string, with a space inserted where the horizontal gap between two characters is more than 1.2 × the median character width.

2.3 Mathematical formulation

Step 1 – Segmentation. The frame is converted to grey, blurred, and thresholded with a local (adaptive) threshold, giving a binary ink mask. Each 8-connected component of the mask whose area and height are within limits is a character candidate with a bounding box. Each candidate is resized to fit a 20×20 box (aspect ratio kept) and centred in a 28×28 image g – the same format as the training data.

Step 2 – Feature vector. A feature map φ turns the 28×28 image into 177 numbers. The first 49 are the average ink in each 4×4 block (ink-density zones); the other 128 are stroke-direction values: the image is cut into 4×4 cells and in each cell the gradient strength is added

up in 8 orientation bins (0°–180°), then the whole vector is scaled to unit length. Written as x = φ(g): 28×28 image in, 177 numbers out.

Step 3 – Fuzzy rule base. For every class k there are 4 fuzzy rules R(k, r) (188 rules in total):

$$\text{IF } x_1 \text{ is } \sim c_1 \text{ AND } x_2 \text{ is } \sim c_2 \text{ AND } \ldots \text{ THEN class} = k$$

Each antecedent "x is ~c" is a Gaussian membership function, where c is the typical value of that feature and σ says how much it varies (every rule has its own c and σ for each of the 177 features):

$$\mu(x) = e^{-(x-c)^2/2\sigma^2}$$

The centres c and widths σ of the rules come from fuzzy c-means clustering (Section 3.2). AND is implemented as the geometric mean of the memberships, so the firing strength of a rule is

$$f = (\mu_1 \cdot \mu_2 \cdots \mu_{177})^{1/177}$$

Rules of the same class are combined with OR = max, giving the membership of the character in class k. Defuzzification is by maximum membership: the class with the largest membership is the answer ĉ.

$$\mu_k = \max\ (f \text{ of the rules of class } k)$$

The strength s is the winning membership, and the gap γ = s − (runner-up membership) shows how far ahead it is.

Step 4 – Reliability controller (Mamdani). Inputs s and γ (both in [0,1]) are fuzzified with triangular sets (Figure 4), nine IF-THEN rules (e.g. IF strength is HIGH AND gap is LARGE THEN reliability is HIGH) are evaluated with AND = min, clipped outputs are joined with max, and the centroid of the aggregated output set gives the reliability ρ between 0 and 100:

$$\rho = \frac{\sum y \cdot \mu(y)}{\sum \mu(y)}$$

2.4 Objective and performance measures

Find the rule parameters that maximise the top-1 accuracy (correct readings ÷ number of test images) on a held-out test set, and make ρ meaningful: samples with higher ρ should be recognised correctly more often. Secondary measure: top-3 accuracy (true class among the three largest class memberships).

2.5 Assumptions and scope

- Dark ink on light paper, one or more lines of separate (non-cursive) characters, reasonably large and well lit.
- Only the 47 classes above; no punctuation. Touching characters are not split.
- Recognition is character-by-character; there is no dictionary or language model.

3. Methodology

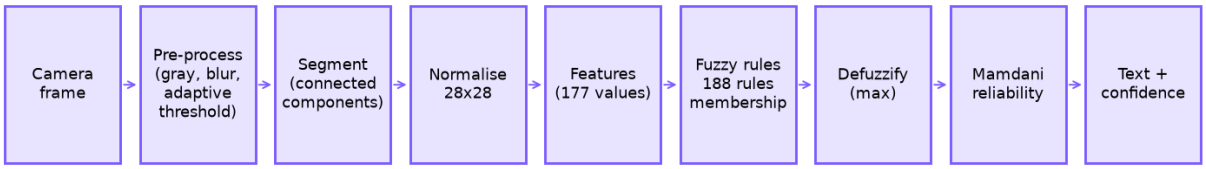


Figure 1: System block diagram

3.1 Dataset

EMNIST-balanced (Cohen et al., 2017): 112,800 training and 18,800 test images, 28×28, 47 classes with equal numbers of samples. It is derived from NIST handwriting and is the letters-and-digits extension of MNIST.

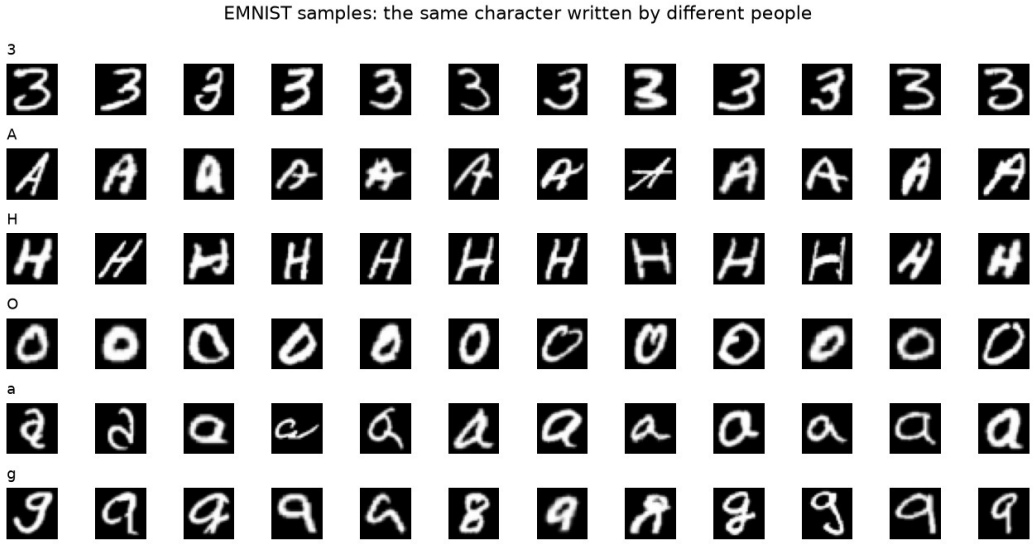


Figure 2: EMNIST samples. Each row is one character class written by different people, so the same character varies a lot in shape and thickness.

3.2 Learning the fuzzy rules with fuzzy c-means

For each class, its training feature vectors are clustered by fuzzy c-means (FCM, fuzzifier 2, 4 clusters). Unlike k-means, every sample belongs to every cluster with a degree u between 0 and 1, and the degrees of one sample add up to 1. FCM repeats two steps until it settles: (a) each cluster centre is the average of the samples weighted by u^2, (b) each membership u is recomputed from the distances d of the sample to the cluster centres.

$$c = \frac{\sum u^2 x}{\sum u^2}$$

Each cluster becomes one rule: its centre gives c and the membership-weighted standard deviation gives σ. A floor of 0.3 × the overall standard deviation of that feature is applied, so a rule is never unrealistically narrow. The idea behind several rules per class is that one letter has several writing styles (e.g. 4 vs. open-top 4, 7 with or without a bar). Section 4.2 measures how much this actually helps.

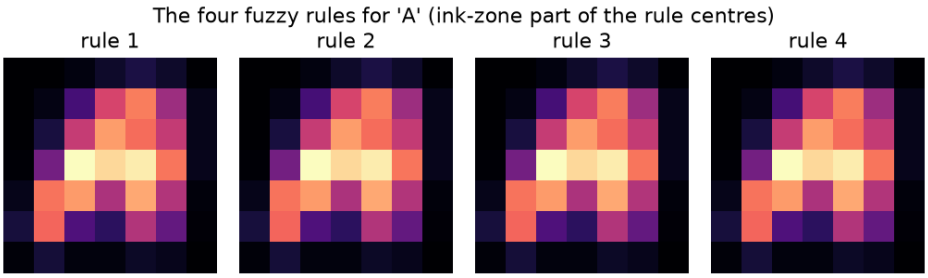


Figure 3: the four fuzzy rules learned for the letter A (ink-zone part of each rule centre). The four rules turn out to be very similar to each other, which matches the experiment in Section 4.2 where extra rules help only a little.

3.3 Reliability fuzzy controller

Strength \ Gap	SMALL	MEDIUM	LARGE
LOW	LOW	LOW	MEDIUM
MEDIUM	LOW	MEDIUM	HIGH
HIGH	MEDIUM	HIGH	HIGH

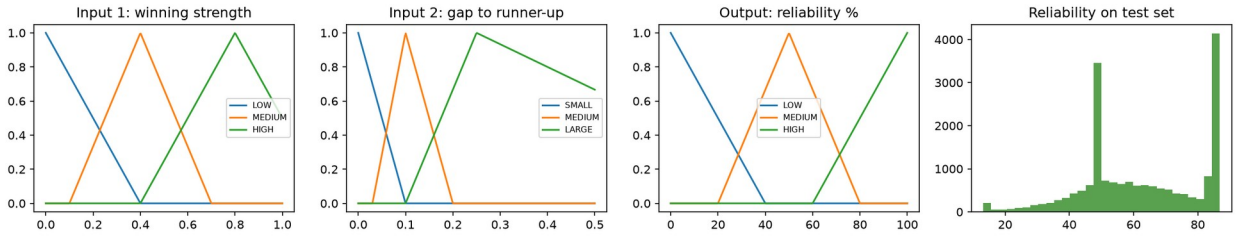


Figure 4: membership functions of the reliability controller and the distribution of ρ on the test set

3.4 Camera application

A Tkinter desktop application reads frames with OpenCV, runs the pipeline of Figure 1 about 6 times per second, and draws a box around each character, coloured green/amber/red by reliability level (≥65 / 35–65 / <35 %). The side panel shows the detected text, the average reliability bar, and for each character the three best classes with their membership values. Buttons: Freeze/Resume, Save snapshot, Open image…, Switch camera.

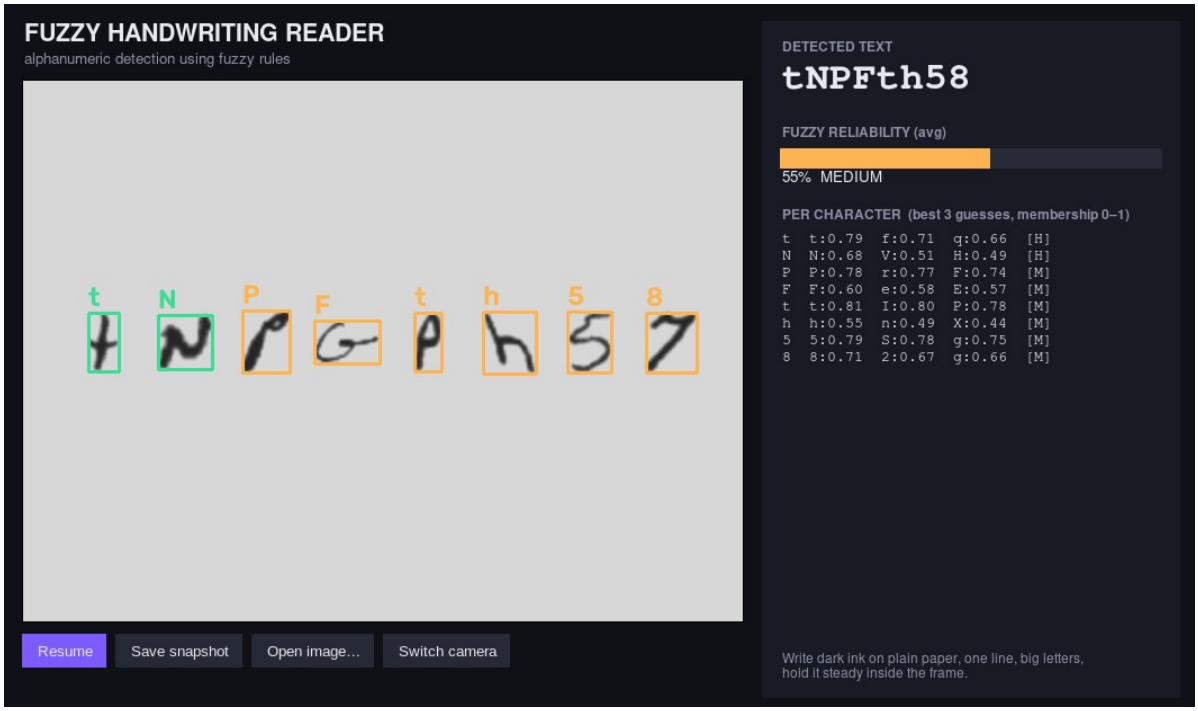


Figure 5: the Tkinter application reading a still image (Open image… mode). In camera mode the left panel shows the live feed instead. Box colour = reliability level; the panel on the right lists the three best classes and their membership for each character.

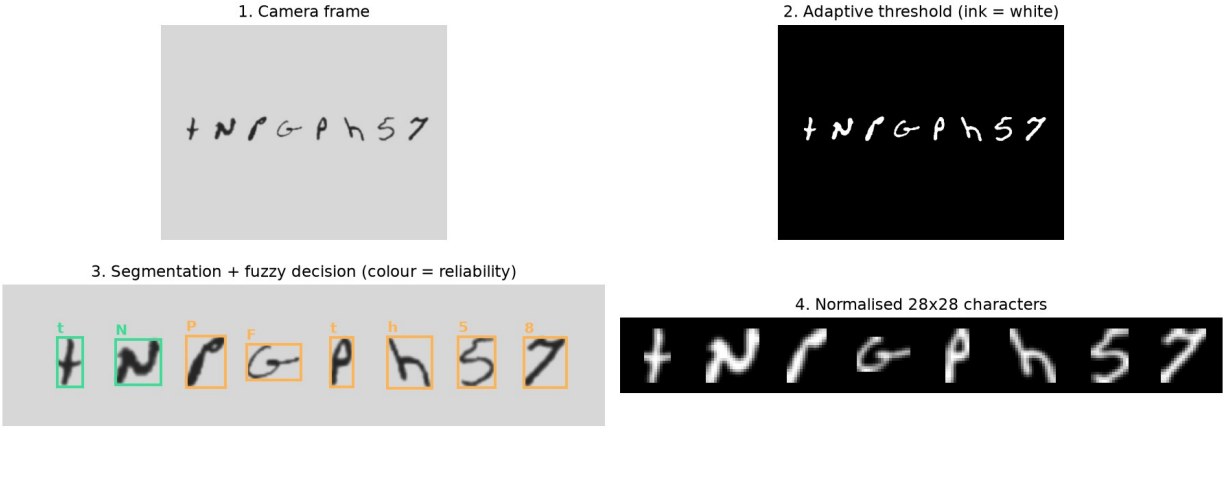


Figure 6: pipeline on a sample image (characters taken from the EMNIST test set and placed on a paper-like background)

4. Results (progress till date)

4.1 Accuracy and reliability

Measure	Value
Top-1 accuracy (18,800 test images, 47 classes)	66.3%
Top-3 accuracy	86.8%
Number of fuzzy rules	188
Random-guess baseline	2.1 %

Does the reliability value mean anything? Test samples grouped by the reliability the controller assigned:

Reliability group	Samples	Accuracy in group
LOW (<35)	1096	28.6%
MEDIUM (35-65)	9661	51.5%
HIGH (>=65)	8043	89.2%

Accuracy rises from the LOW to the HIGH group, so ρ is a useful warning: when the fuzzy controller says HIGH the reading is right about nine times out of ten.

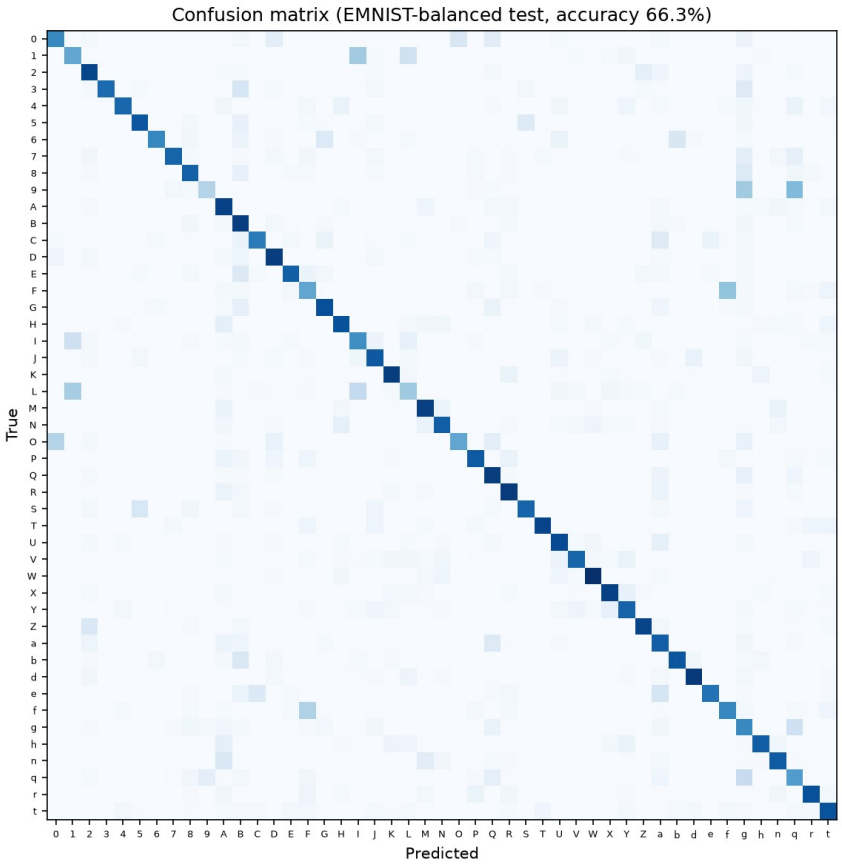


Figure 7: confusion matrix

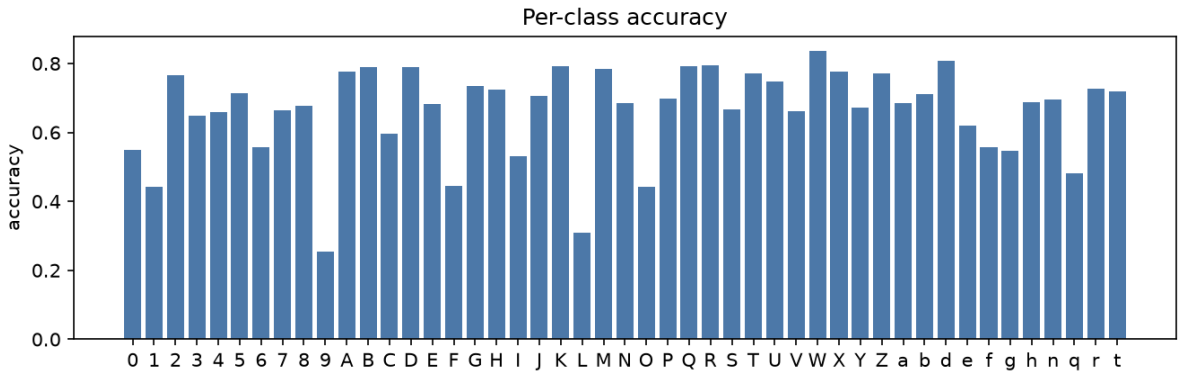


Figure 8: accuracy per class

Observations. Most errors are between look-alike shapes: 9/q/g, F/f, 1/I/L, O/0, e/a and 3/B (the most frequent confusions in our test run). This is the inherent imprecision that motivated the fuzzy approach; the top-3 accuracy shows that the correct character is almost always among the top candidates.

4.2 Experiments: what actually matters?

To understand which design choices matter, we changed one thing at a time and measured test accuracy.

Rules per class	Test accuracy
1	65.9%
2	66.2%
4	66.3%
8	66.5%

Features used (4 rules per class)	Test accuracy
ink zones only (49)	50.2%
stroke directions only (128)	65.5%
both (177)	66.3%

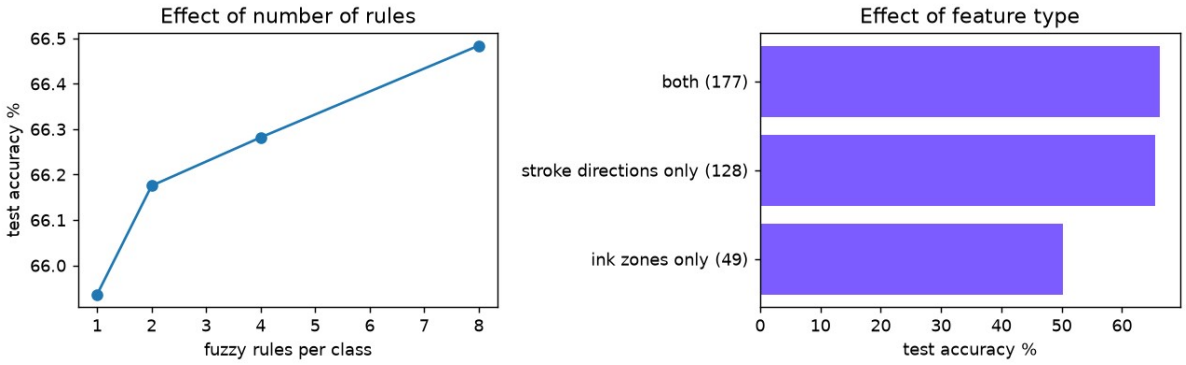


Figure 9: effect of the number of rules and of the feature type

Findings. (1) The stroke-direction features carry most of the information: using them alone gives 65.5%, while the ink zones alone give only 50.2%. Adding the ink zones to the

directions improves the result slightly. (2) More rules per class helps only a little (65.9% with one rule, 66.5% with eight), so the limit is the feature description and the simple Gaussian membership form, not the number of writing styles. We kept 4 rules per class as a compromise between accuracy and the size of the rule base. These findings point to where improvement should come from next (better features, tuned membership widths).

5. Limitations and future work

- Accuracy (about 66 %) is lower than a trained neural network would reach (CO3). The fuzzy rules are deliberately simple and explainable: every decision can be traced back to rules and membership values.
- Future: tune the rule widths and number of rules per class with a genetic algorithm (CO4); add a perceptron / neural network and compare; use context (dictionary) to resolve O/0 and I/1; split touching characters.
- Camera robustness: strong shadows, thin pen strokes and tilted paper reduce quality.

6. Tools and how to run

Python 3, NumPy (fuzzy c-means and fuzzy rules written from scratch), OpenCV (image processing), Tkinter + Pillow (GUI), Matplotlib (figures). Commands: python src/train.py | python src/evaluate.py | python src/app.py [camera index or image file]

7. References

- L. A. Zadeh, “Fuzzy sets”, Information and Control, 8(3), 1965.
- J. C. Bezdek, Pattern Recognition with Fuzzy Objective Function Algorithms, Plenum, 1981 (fuzzy c-means).
- E. H. Mamdani and S. Assilian, “An experiment in linguistic synthesis with a fuzzy logic controller”, 1975.
- G. Cohen, S. Afshar, J. Tapson, A. van Schaik, “EMNIST: an extension of MNIST to handwritten letters”, 2017.
- S. Rajasekaran and G. A. Vijayalakshmi Pai, Neural Networks, Fuzzy Logic and Genetic Algorithms, PHI.
- N. Dalal and B. Triggs, “Histograms of oriented gradients for human detection”, CVPR 2005 (idea behind direction features).